সংশোধন: লুপ, জমা এবং আকৃতির মেনু

পাঠ 14 — 1IOT, SST-ICT-01 — কাজ নিয়ে প্রশ্ন, জমার নিয়ম, কাজ C01-এর অনুশীলন 6

26/10/2026

1 আজ আমরা কী করলাম

আজ কোনো নতুন বিষয় শুরু করিনি: এটি ছিল **সংশোধনের** পাঠ। শুরুতে আমি বোর্ডে বাড়ির কাজ ও তোমাদের প্রশ্ন থেকে আসা **অনুরোধের তালিকা** লিখেছি:

- **ইন্ডেন্টেশন** (অংশ 2);
- **while** (অংশ 3);
- ফাঁকের বদলে **বিন্দু** (অংশ 4);
- **do while** (অংশ 5);
- অনুশীলনের **ভুলের একটি বিবরণ** (অংশ 6 ও 7)।

তারপর ক্লাসে তোমরা বাড়ির কাজের **যাচাই** করেছ, আর দ্বিতীয় অংশে আমরা **একসাথে সংশোধন** করেছি কাজ C01-এর **অনুশীলন 6**, **আকৃতির মেনু** (অংশ 8), এই ধারার সবচেয়ে কঠিনটি। আজকের জন্য ঠিক করা ফাংশন ও তার প্যারামিটার পরের পাঠগুলোতে দেখব।

2 ইন্ডেন্টেশন

ইন্ডেন্টেশন (**indentatura** বা **indentazione**) হলো নির্দেশগুলো **সরিয়ে** লেখার উপায়, অর্থাৎ সব বাঁ প্রান্তে সারিবদ্ধ নয়: যেমন `if (x == 3)` আর খোলা বন্ধনীর পরে `printf` বন্ধনীর নিচে লিখি না, বরং **ডানে সরিয়ে**, প্রান্ত ও লেখার শুরুর মাঝে একটি খালি জায়গা রেখে।

2.1 C-তে ফলাফল বদলায় না

C-তে ইন্ডেন্টেশনের **চালানোর ক্ষেত্রে কোনো অর্থ নেই**: পুরো প্রোগ্রাম **এক লাইনে** লিখলেও চলত। অন্য ভাষায় তা নয়: **Python**-এ (আরেকটি প্রোগ্রামিং ভাষা, যেমন কিছু উচ্চ বিদ্যালয়ে পড়ানো হয়) কার্লি বন্ধনী নেই, আর যন্ত্র ইন্ডেন্টেশন দিয়েই বোঝে একটি নির্দেশ অন্যটির ওপর **নির্ভরশীল**: একটি `if`-এর নিচে ইন্ডেন্ট করা সব নির্দেশ সেই `if`-এর ভেতরে ধরা হয়।

C-তে এই কাজ করে **কার্লি বন্ধনী**। বোর্ডের উদাহরণ দেখি:

```c
int main()
{
    int x;
    x = 3;
    if (x == 3)
    {
        printf("Inserisci un numero: ");
        scanf("%d", &n);
        printf("...");
    }
    printf("...");
}
```

বন্ধনীর ভেতরের তিনটি নির্দেশ (`printf`, `scanf`, `printf`) **তিনটিই নির্ভর করে** `x` 3-এর সমান কিনা তার ওপর: `x` 3 হলে তিনটিই চালায়, `x` 7 হলে **বাদ দেয়**। বন্ধ বন্ধনীর পরের `printf` সবসময় চলে। বন্ধনীর তৈরি **আধার** কোডের একটি অংশকে সীমিত করে **একটিমাত্র নির্দেশের মতো** ধরে।

সংজ্ঞা

ইন্ডেন্টেশন একটি বন্ধনীর ব্লকের **ভেতরের** নির্দেশগুলোকে 3–4 অক্ষর ডানে সরায়, যাতে সঙ্গে সঙ্গে দেখা যায় ওই নির্দেশগুলো **একটি আলাদা দল**। C-তে এটি শুধু **পাঠকের** জন্য, যন্ত্রের জন্য নয়।

টীকা

`if`-এর পরের লাইন আগে থেকেই সরানো কেন? কারণ `if` সাধারণত `main`-এর বন্ধনীর **ভেতরেই** থাকে: তার সামনে একটি সরানো আছে, আর `if`-এর ভেতরের অংশ **আরেকটি** পায়। বন্ধনীর প্রতিটি স্তর একটি সরানো যোগ করে।

2.2 Tab কী

সাধারণত ইন্ডেন্ট করতে **ট্যাব** ব্যবহার হয়, অর্থাৎ **Tab** কী (দুটি তীরের কী, Caps Lock-এর ওপরে, বাঁয়ে)। এটি স্বয়ংক্রিয়ভাবে লেখাকে কয়েক ফাঁক সরায় এবং **একটি অক্ষর হিসেবে** ধরে: ট্যাব দিয়ে পেছনে ফিরলে কার্সর পুরো এক ধাপ ফেরে, মাঝপথে থামে না।

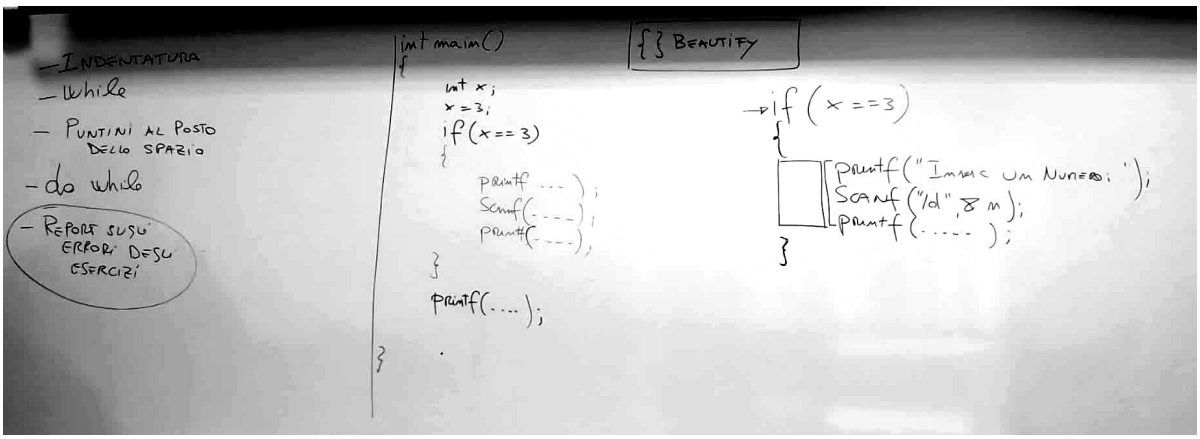
ট্যাব কত চওড়া তা **এডিটরের** ওপর নির্ভর করে: Windows-এর Notepad 8 অক্ষর চওড়া দেখায় (ক্লাসে বলেছিলাম 5–6), অন্য এডিটর 2, 3 বা 4। onlinegdb-তে 4 হওয়ার কথা। আমাদের কিছু যায় আসে না: জরুরি হলো এটি ব্যবহার করা।

2.3 Beautify বোতাম

ব্লকের ভেতরে নতুন লাইনে গেলে onlinegdb নিজেই কার্সর **ঠিক জায়গায়** রাখে। এছাড়া onlinegdb-তে (ও অন্য যেসব প্রোগ্রাম ব্যবহার করব) একটি বোতাম আছে যা **স্বয়ংক্রিয়ভাবে** সঠিক ইন্ডেন্টেশন করে: এর নাম **Beautify**, ওপরে আছে, দুটি কার্লি বন্ধনীর প্রতীক `{}` সহ।

সতর্কতা

তোমাদের মাথায় কী ছিল Beautify **জানে না**: **যা লেখা আছে** তা দেখে নিয়ম মেনে ইন্ডেন্ট করে। একটি বন্ধনী ভুল হলে সেও ভুল ইন্ডেন্ট করে, আর তাতে তার দোষ নেই। ঠিক এজন্যই এটি কাজের: Beautify-এর পরে `if`-এর বাইরে থাকার কথা এমন একটি `printf` **ভেতরে সরে গেলে**, বা শেষ বন্ধনী একা থেকে গেলে, তোমরা **ভুলটি** পেয়ে গেছ।



ছবি/নকশা 1: পাঠের শুরুর বোর্ড: অনুরোধের তালিকা, `main`-এর ভেতরে ইন্ডেন্ট করা `if`-এর উদাহরণ এবং Beautify বোতাম। ডানে `if`-এর ওপর নির্ভরশীল তিনটি নির্দেশের ব্লক।

2.4 কৌশল: আগে গঠন

বন্ধনী না ভুলতে একটি কৌশল সবসময় কাজ করে: কোড লেখার আগে **গঠনটি লেখো**।

1. `if`, খোলা গোল বন্ধনী, বন্ধ গোল বন্ধনী লিখি।
2. নতুন লাইনে যাই, **খোলা কার্লি বন্ধনী**, **বন্ধ কার্লি বন্ধনী**।
3. তারপরই বন্ধনীর ভেতরে ঢুকে **ফাঁকা জায়গাগুলো ভরি**: শর্ত, `printf`, `scanf`।

আর `if (x == 3)` লিখে বন্ধনী খুলে তারপর `printf`, `scanf` ইত্যাদি লিখলে, পথে **বন্ধ বন্ধনী ভুলে যাওয়া** সহজ।

2.5 প্রোগ্রাম চললেও কেন করব?

কেউ বলতে পারে: প্রোগ্রাম চললে ইন্ডেন্টেশন নিয়ে ভাবব কেন? আমি বুঝি, অন্যরা সামলে নিক। এভাবে চলে না:

- প্রোগ্রামারের কাজ **একার কাজ নয়**: কাজ হয় **দলে**, আর কোড অন্যরাও পড়ে ও বদলায়;
- একা যে কাজ করে তাকেও একদিন **তিন বছর আগে** লেখা কোড **আবার হাতে নিতে** হয়: সব মনে রাখা সম্ভব নয়।

তাই ইন্ডেন্টেশন **মৌলিক**, যদিও ফলাফল বদলায় না।

উদাহরণ

পরীক্ষার প্রশ্ন। ইন্ডেন্টেশন কি প্রোগ্রামের ফলাফলকে প্রভাবিত করে?
উত্তর: না। এটি শুধু **পাঠযোগ্যতাকে**, অর্থাৎ কোডের দৃশ্যমান সংগঠনকে প্রভাবিত করে।

3 while: এটি আসলে কী করে

«while কী করে?» প্রশ্নে ক্লাসে উত্তর এসেছে যেমন «ঠিক না হওয়া পর্যন্ত পুনরাবৃত্তি করে» বা «সঠিক ডেটা না দেওয়া পর্যন্ত প্রশ্ন আবার করে»। এই উত্তরগুলো আমরা যে **উদাহরণ** করছিলাম (ডেটা যাচাই) তার সাথে যুক্ত, while **কী** তার সাথে নয়।

সংজ্ঞা

while একটি **গঠন**, অর্থাৎ প্রোগ্রামিংয়ের একটি কাঠামো, যা কোডের একটি অংশকে **একটি শর্ত সত্য থাকা পর্যন্ত পুনরাবৃত্তি** করতে দেয়। «যতক্ষণ একটি শর্ত আছে» নয়: যতক্ষণ শর্ত **সত্য**। **মিথ্যা** হলে থামে।

3.1 আলমারির উদাহরণ

আমি একটি যন্ত্র, একজন সহকারী প্রোগ্রাম করি এবং বলি: «আলমারিতে না পৌঁছানো পর্যন্ত **হাঁটো**»। সে এক পা ফেলে জিজ্ঞেস করে: আলমারিতে পৌঁছেছি? না। তাই আবার করে। আরেক পা: পৌঁছেছি? না। এভাবে চলে। কত পা ফেলবে? জানা নেই: কিন্তু **প্রতি পায়ে যাচাই করে** পৌঁছেছে কিনা। উত্তর বদলালে হাঁটা **থামায়**।

সতর্কতা

সঠিকভাবে বললে, সঠিক প্রশ্নটি **উল্টো**: «আলমারিতে এখনো পৌঁছাইনি?»। উত্তর যতক্ষণ **সত্য** (হ্যাঁ, সত্য যে পৌঁছাইনি), আরেক পা ফেলি। পৌঁছালে «পৌঁছাইনি» **মিথ্যা** হয়, আর আমি থামি। while-এ সবসময় **সত্য শর্তই** পুনরাবৃত্তি করায়।

3.2 while ও for একই জিনিস

```c
int n = 0;
while (n < 10)
{
    // ... নির্দেশ ...
    n++;
}
```

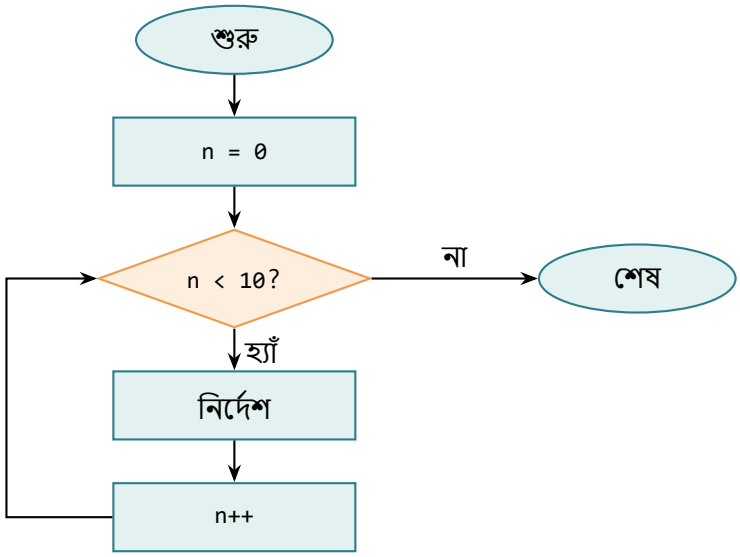
```c
for (int n = 0; n < 10; n++)
{
    // ... নির্দেশ ...
}
```

ছবি/নকশা 2: একই লুপ while (বাঁয়ে) ও for (ডানে) দিয়ে লেখা, বোর্ডের মতো।

while-এ গোল বন্ধনীর ভেতরে থাকে **পুনরাবৃত্তির শর্ত**: `n < 10`, `n != 7`, যেকোনো শর্ত, আমরা কী করছি তার ওপর নির্ভর করে। ব্লকের ভেতরে থাকে **পরিবর্তক**: `n++`, `n = n * 2`, `n = n + 2`, `n = n * 3 - 4 +`

`18 + k`, যা খুশি। জরুরি হলো **n** **বদলানো**, নাহলে শর্ত চিরকাল সত্য থাকে।

for একই কাজ করে: **মাঝে** দেওয়া শর্ত যতক্ষণ সত্য, পুনরাবৃত্তি করে। পার্থক্য শুধু লেখায়: for-এ **শুরুর মান**, **শর্ত** ও **পরিবর্তক** তিনটিই গোল বন্ধনীর ভেতরে।



ছবি/নকশা 3: দুটি লুপের সাধারণ ফ্লোচার্ট: প্রতিটি পুনরাবৃত্তির **আগে** শর্ত যাচাই হয়।

3.3 কখন for আর কখন while

এটি **শৈলীর** ব্যাপার। সাধারণত:

- **for** ব্যবহার হয় যখন **আগে** **থেকে** জানা কতবার পুনরাবৃত্তি: প্রতিবার **দশটি** **লাফের** একটি অনুশীলন; জানি দশটিই;
- **while** ব্যবহার হয় যখন **জানা** **নেই** কতবার: **ক্লান্ত** **না** **হওয়া** **পর্যন্ত** লাফানো নির্ভর করে কতটা সহ্য করা যায়।

এটি শুধু পছন্দ: নির্দিষ্ট সংখ্যায় while বা না জেনে for ব্যবহারে বাধা নেই, শুধু প্রোগ্রামিংয়ের **অলিখিত নিয়ম** ছাড়া, যা সবাই মানে, অনেকটা ভদ্রতার নিয়মের মতো।

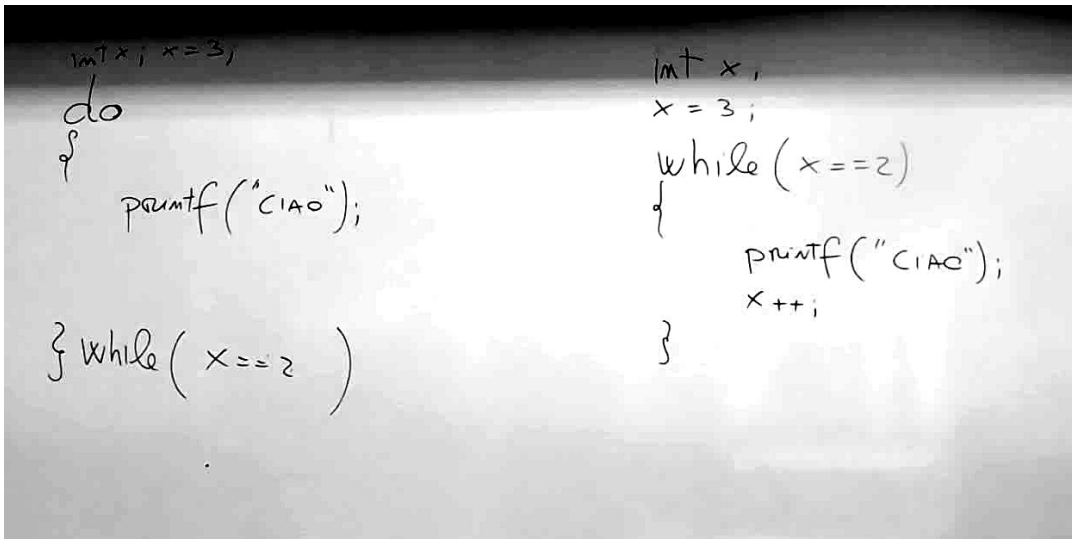
4 ফাঁকের বদলে বিন্দু

আকৃতির অনুশীলনে কিছু জায়গায় তারকাচিহ্ন ছাপা হয়, `printf("*");`, অন্য জায়গায় একটি **ফাঁক**, `printf(" ");`। অনুশীলনের লেখায় ফাঁকের বদলে ছিল **বিন্দু**। কারণটি লেখাতেই ছিল: ফাঁক **দেখা** **যায়** **না**, আর একটি জায়গায় ফাঁক আছে কিনা বোঝা কঠিন। বিন্দু শুধু **ফাঁকের** **জায়গা** **নেয়**, যাতে তা দেখা যায়।

প্রোগ্রামে, আঁকায় যেখানে বিন্দু, সেখানে **ফাঁক** ছাপা হয়। তবে বিন্দু ছাপালেও সমস্যা নেই: অনুশীলনের লক্ষ্য হলো পুনরাবৃত্তির **লুপ**, `if` ও **নেস্টিংয়ের** **অনুশীলন**। তারকাচিহ্নের বদলে `#`, `@` বা যা খুশি দিতে পারো: জরুরি হলো লুপ ঠিকভাবে করা।

5 do while ও while: একমাত্র পার্থক্য

do while বন্ধনীর ব্লক পুনরাবৃত্তি করে যতক্ষণ **শেষে** লেখা শর্ত সত্য। ব্লকের ভেতরে **যা খুশি** থাকতে পারে: অনুশীলনে প্রায় সবসময় একটি `printf` ও একটি `scanf` ছিল, কিন্তু সবসময় শুধু তাই থাকবে এমন নয়। এ পর্যন্ত while-এর মতোই। **একটিমাত্র পার্থক্য** আছে: while প্রথমবারেই **কিছু না করতে** পারে, do while পারে না।



ছবি/নকশা 4: বোর্ডের উদাহরণ: বাঁয়ে do while, ডানে while, একই শর্ত `x == 2` আর `x`-এর মান 3।

```c
int x;
x = 3;
do
{
    printf("ciao");
} while (x == 2);
```

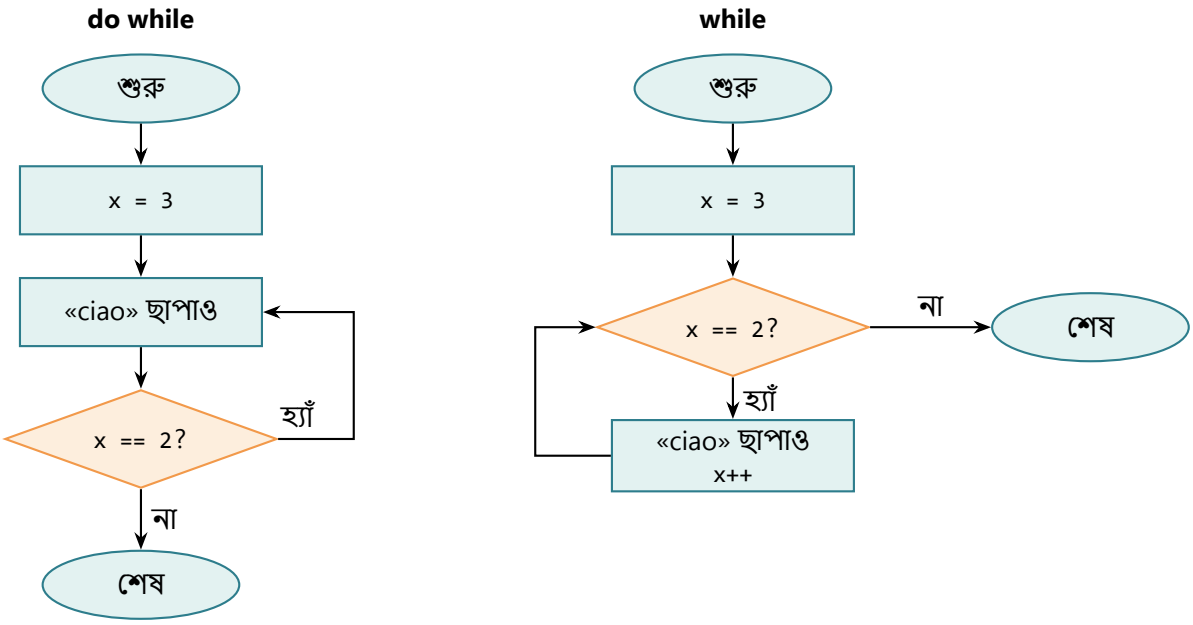
```c
int x;
x = 3;
while (x == 2)
{
    printf("ciao");
    x++;
}
```

উদাহরণ

«ciao» কতবার ছাপে?

- **while** (ডানে): x 3 আর while বলে «x 2-এর সমান যতক্ষণ পুনরাবৃত্তি করো»। শর্ত শুরুতেই **মিথ্যা**: সঙ্গে সঙ্গে শেষে চলে যায় এবং **কিছু ছাপে না**। **শূন্য বার।**
- **do while** (বাঁয়ে): শর্ত একই, সবসময় মিথ্যা, কিন্তু তা যাচাইয়ে পৌঁছাতে প্রোগ্রামকে **ব্লক পেরোতে** হয়: তাই একটি «ciao» ছাপে। **একবার।**

do while মানে যেন **একবার** `printf("ciao")` লিখে তারপর while করা।



ছবি/নকশা 5: do while-এ ছাপা হয় প্রশ্নের **আগে**, while-এ **পরে**: `x = 3` হলে প্রথমটি একবার ছাপে, দ্বিতীয়টি কখনো নয়।

টীকা

do while কখন ব্যবহার হয়? ব্লক **অন্তত একবার** চালাতে হলে **do while** সুবিধাজনক (যেমন ব্যবহারকারীকে একটি প্রশ্ন, যা অন্তত একবার করতে হয়)। while দিয়েও একই ফল পাওয়া যায়, কিন্তু লুপের আগে ভেরিয়েবলগুলো একটু «চালাকি করে» প্রস্তুত করতে হয়। ব্লক চলবে কিনা জানা না থাকলে **while** ব্যবহার হয়।

টীকা

Polling। ক্লাসে মাঝে মাঝে জিজ্ঞেস করি «**polling?**»: এটি একটি **কৌশলের** নাম, যাতে **নির্দিষ্ট সময় পরপর জিজ্ঞেস করা হয়** নতুন কিছু আছে কিনা; কেউ উত্তর না দিলে মানে কিছু নেই। তোমরা ফোনে প্রতি মিনিটে বার্তা এসেছে কিনা দেখে এটাই করো। ক্লাসে মানে: «তোমরা আছ? প্রশ্ন আছে?»

6 বাড়ির কাজের জমা

বাড়ির কাজ **মূল্যায়ন করা হয়**, আর ক্লাসের যাচাই বোঝার জন্য যে সেগুলো **তোমরা নিজে করেছ** কিনা। যাচাইয়ে যদি দেখা যায় সব ঠিক জমা দেওয়া একটি কাজ জমাদাতা করেনি, মূল্যায়ন অনেক **কমে যায়**। নিজের ভুল ঠিক করা ভালো: **সংশোধন** মানে যা ভুল হয়েছে তা ঠিক করা। যে কাজ ব্যাখ্যা করতে পারো না তা জমা দেওয়া অন্য জিনিস।

6.1 শেষ সময় মানে শেষ সময়

জমার একটি **সময়** আছে: শেষ সময় 20:00 হলে শেষ মুহূর্ত **19:59:59**। 20:02 মানে **দেরি**।

উদাহরণ

সরকারি দরপত্র। মাঝে মাঝে রাষ্ট্র, অঞ্চল বা পৌরসভা একটি **দরপত্র** (**bando**) প্রকাশ করে: একটি প্রকাশ্য অনুরোধ, যাতে তারা বলে «এই কাজ করার মতো কাউকে খুঁজছি, দরপ্রস্তাব পাঠান»; সেরা দরপ্রস্তাব যে দেয় সে কাজ পায়। তুমি কোনো কোম্পানিতে কাজ করলে এবং দরপত্র আকর্ষণীয় হলে, এটি আয়ের একটি উপায়। কিন্তু আবেদন শেষ সময়ের **এক মিনিট পরেও** এলে **বিবেচনাই করা হয় না**: কাজ হারিয়ে গেল। জমার সময় **সবসময় মানতে** হবে।

6.2 পরের কাজ

অনুশীলনের নতুন প্যাকেট (3–5টি অনুশীলন) Classroom-এ আসবে এবং জমা দিতে হবে **শনিবারের মধ্যে**, যাতে রবিবার সংশোধনের সময় থাকে। সমস্যা থাকলে (কম্পিউটার নেই, সংযোগ নেই), এটি **অজুহাত নয়**: শেষ সময়ের আগে জমা দাও, অথবা কারও কাছ থেকে কম্পিউটার ধার নাও, বন্ধুর বাড়িতে, এবং শনিবার সন্ধ্যার মধ্যে ইমেইলে কাজ পাঠাও। কীভাবে তা জরুরি নয়: জরুরি হলো **পৌঁছানো**।

সতর্কতা

বাড়ির কাজ **বাধ্যতামূলক**। যে উল্টো বলে সে **ভুল** কথা বলে।

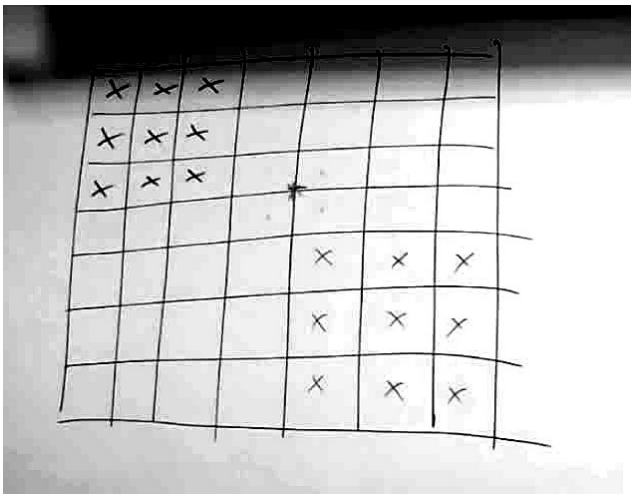
7 ছকে আউটপুট আঁকা

ক্লাসের যাচাইয়ে প্রত্যেকের ছিল একটি **আলাদা** অনুশীলন (আয়তক্ষেত্র, বর্গ, ত্রিভুজ...) আর এই ধরনের একটি প্রশ্ন:

উদাহরণ

«ব্যবহারকারী বাহু 3 লিখলে প্রোগ্রাম কী ছাপে? নিচে আঁকো: **প্রতি ঘরে একটি অক্ষর**, **ফাঁকগুলো খালি রাখো**।»

অনেক আঁকার আকৃতি ঠিক ছিল, কিন্তু **ভুল জায়গায়**: যেমন ছকের **মাঝখানে** আঁকা একটি 3×3 আয়তক্ষেত্র, আগে চারটি খালি ঘরসহ।



ছবি/নকশা 6: বোর্ডে: ওপরে বাঁয়ে সঠিক 3×3, আর একই 3×3 ছকের মাঝে সরানো, যা প্রোগ্রাম কখনো ছাপে না।

কোণে «দেখতে খারাপ» বলে কম্পিউটার আঁকা **সরায় না**: সে **লাইনের শুরু থেকে**, অর্থাৎ **ওপরে বাঁ দিক থেকে** ছাপে, আর আমরা যে ফাঁক ও নতুন লাইন ছাপাতে বলেছি শুধু তা দিয়ে এগোয়। **কম্পিউটারের মতো** ভাবতে শিখতে হবে, যে **ব্যাখ্যা করে না**: যা বলি তাই চালায়।

আউটপুট আঁকার নিয়ম:

1. শুরু **ওপরে বাঁয়ের** ঘর থেকে;
2. **প্রতি ঘরে একটি অক্ষর**, ঘরের **ভেতরে** লেখা, রেখার **সংযোগে নয়**, নাহলে আশপাশের চার ঘরের কোনটিতে দিতে চেয়েছ বোঝা যায় না;
3. প্রোগ্রাম যেখানে ফাঁক ছাপে সেখানে ঘর **খালি** থাকে: কাজের নির্দেশে «ফাঁক খালি রাখো» বলা থাকলে বিন্দু নয়।

টীকা

স্যান্ডউইচ। আমি বেগুনের স্যান্ডউইচ চাই আর অন্য কিছু দিয়ে স্যান্ডউইচ পাই: কেন? বেগুন না থাকলে জিজ্ঞেস করা হয়, নিজে থেকে অনুরোধ বদলানো হয় না। জমাতেও একই: **যা লেখা আছে** তাই করা হয়, সন্দেহ হলে **জিজ্ঞেস করা** হয়। এই ভুলগুলো শাস্তি নয়: এগুলো **খুঁটিনাটিতে মনোযোগ দিতে** শেখায়, কারণ ভুল জায়গায় সঠিক আকৃতিও মূল্যায়নে **ক্ষতি করে**।

8 C01-এর অনুশীলন 6: আকৃতির মেনু

সময় থাকলে কাজের সংশোধন **ক্লাসে** হয়, আজকের মতো; নাহলে শুধু **সংশোধনের ফাইল** পাও। পাঠের দ্বিতীয় অংশে আমরা onlinegdb-তে একসাথে সংশোধন করেছি কাজ C01-এর **অনুশীলন 6** (কাজের নথিতে এটি অধ্যায় 7), ধারার সবচেয়ে লম্বাটি। লেখাটি, প্রোগ্রামের শুরুতে মন্তব্য হিসেবে কপি করা:

উদাহরণ

প্রশ্ন। একটি **মেনুসহ** প্রোগ্রাম লেখো, যা ব্যবহারকারী **0 না বাছা পর্যন্ত** আবার আসে: 1 = বর্গ, 2 = ফাঁপা বর্গ, 3 = ত্রিভুজ, 0 = বের হও।

- 1, 2 বা 3 দিলে **বাহু** চায় (1 থেকে 9, **যাচাইসহ**) এবং আকৃতি আঁকে: ভরা বর্গ, ফাঁপা বর্গ (শুধু সীমানা), সমকোণী ত্রিভুজ;
- মেনুতে নেই এমন সংখ্যায় ছাপে **Scelta non valida**;
- প্রোগ্রাম **গোনে** কয়টি আকৃতি এঁকেছে; 0 বাছলে ছাপে `Figure disegnate:` ও সংখ্যাটি।

ইঙ্গিত: একটি **do while**-এর ভেতরে একটি **switch**, পাঠ 12-এর মেনুর মতো। কাউন্টার লুপের আগে 0 থেকে শুরু হয় এবং প্রতিবার আকৃতি আঁকলে 1 বাড়ে (`figure++`)। তিনটি আঁকা তিনটি আলাদা `case`-এ লিখতে পারো, আগে করা প্রোগ্রাম কপি করে।

8.1 মেনুর লুপ: do while

মেনু **পুনরাবৃত্তি** হবে এবং প্রশ্ন **অন্তত একবার** করতে হবে: **do while** লাগে। ব্যবহারকারী 0 বাছলেই শুধু বের হওয়া যায়, তাই পছন্দ **0 থেকে আলাদা** যতক্ষণ পুনরাবৃত্তি: `while (scelta != 0)`।

লুপের ভেতরে একটি `printf` দিয়ে পছন্দ চাওয়া হয়, যাতে **বিকল্পগুলোও** লেখা ভালো, যেন ব্যবহারকারী জানে কী বাছতে পারে (**Fai una scelta** = একটি পছন্দ করো), আর `scanf` দিয়ে নেওয়া হয়।

```c
int scelta;

do
{
    printf("Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): ");
    scanf("%d", &scelta);

    // … এখানে আকৃতিগুলো …

} while (scelta != 0);
```

8.2 পছন্দ: switch

লেখায় **switch**-এর ইঙ্গিত আছে, «পাঠ 12-এর মেনুর মতো»। এমন ইঙ্গিত পেলে প্রথম কাজ হলো **A12 পাঠ্য খুলে** মেনুর উদাহরণ খোঁজা: সেখানে ঠিক দরকারি গঠনটি আছে। `if` দিয়েও করা যেত, কিন্তু **বহু পছন্দের** জন্য switch বেশি **মার্জিত**, কারণ সংক্ষিপ্ত করে।

switch হয় যে ভেরিয়েবলে পছন্দ রেখেছি তার ওপর, অর্থাৎ `scelta`। ক্ষেত্রগুলো:

- `case 1:`, `case 2:`, `case 3:` — তিনটি আকৃতি;
- `case 0:` — **সমাপ্তি** (আঁকা আকৃতির সংখ্যা ছাপা হয়);
- `default:` — **বাকি সব** সংখ্যা: «Scelta non valida»।

`case 1`-এর পরের **কোলন** গঠনের লেখার অংশ, প্রতিটি ক্ষেত্রের শেষের `break`-এর মতো।

8.3 বাহু: কোথায় চাইব?

1, 2 ও 3-এর জন্য বাহু চেয়ে **যাচাই করতে** হবে (1 থেকে 9)। তিনটি সম্ভাবনা:

1. `printf`, `scanf` ও যাচাইয়ের do while **তিনবার** লেখা, প্রতিটি `case`-এর ভেতরে: এড়াতে হবে। একই জিনিস তিনবার লিখলে **তিনবার ভুলের ঝুঁকি**, আর কিছু বদলাতে হলে **তিনবার** বদলাতে হয়;
2. **switch-এর আগে** একটি `if` রাখা, যা পছন্দ 1, 2 বা 3 হলেই বাহু চায়: এটি **পছন্দের** সমাধান;
3. `case 1:`, `case 2:` ও `case 3:` **পরপর** রাখা, `break` ছাড়া: করা যায়, কিন্তু তারপর আকৃতি আলাদা করতে নিচে একটি `if` লাগবেই, আর কোড কম পরিষ্কার হয়।

দ্বিতীয় সমাধানে: পছন্দ 1 থেকে 3 হলে বাহু চাওয়া হয়; 0 (বের হও) বা অবৈধ সংখ্যা হলে বাহু লাগে না।

```
if (scelta >= 1 && scelta <= 3)
{
    do
    {
        printf("Inserisci il lato (da 1 a 9): ");
        scanf("%d", &lato);
    } while (lato < 1 || lato > 9);
}
```

সতর্কতা

ক্লাসে প্রথম চেষ্টায় do while-এর জন্য পছন্দের শর্ত মাথায় এসেছিল (`scelta >= 1 && scelta <= 3`): এটি ভুল, কারণ সেটি আমরা `if`-এ ব্যবহার করে ফেলেছি। do while-এর শর্ত **বাহু** নিয়ে, পছন্দ নিয়ে নয়: **বাহু ভুল হলে** প্রশ্ন আবার হয়, অর্থাৎ `lato < 1 || lato > 9`। 1 থেকে 9 চাইলে আর ব্যবহারকারী 18 লিখলে প্রশ্ন আবার হয়।

অনুশীলনটি অন্যভাবেও করা যায়, এবং তা ভুল নয়: কিন্তু অনুশীলনের ধারা **switch, for ও while**-এর ওপর গড়া, আর এগুলো দিয়েই করতে হবে। এখন জটিল মনে হলে: তিন মাস পরে পেছনে তাকালে সহজ মনে হবে।

8.4 ক্ষেত্র 1: ভরা বর্গ

দুই মাত্রার আঁকায় **দুটি নেস্টেড for** লাগে: বাইরেরটি সারিতে (`y`), ভেতরেরটি কলামে (`x`)। বর্গে দুটোই `lato` (বাহু) পর্যন্ত যায়, কারণ বাহুগুলো সমান।

```
case 1:
    for (int y = 0; y < lato; y++)
    {
        for (int x = 0; x < lato; x++)
        {
            printf("*");
        }
        printf("\n");
    }
    nFigure++;
    break;
```

x-এর for **এক সারি** তৈরি করে; এই সারি পুনরাবৃত্তি হয় y 0, 1, 2, 3, 4-এর জন্য (বাহু 5 হলে): 4-এ থামে কারণ 0 থেকে শুরু। `printf("*")` ভেতরের for-এর একমাত্র নির্দেশ: বন্ধনী লাগত না, কিন্তু **আমরা সবসময় দিই**। `for(...)`-এর বন্ধ গোল বন্ধনীর পরে সেমিকোলন **দেওয়া হয় না**: সেখানে ; মানে «একটি খালি নির্দেশ পুনরাবৃত্তি করো», আর বন্ধনীর ব্লক একবারই চলত।

টীকা

টুকরো টুকরো পরীক্ষা। শুধু `case 1` লিখে **শুধু 1** বেছে প্রোগ্রাম পরীক্ষা করা হয়: 2 ও 3 এখনো সামলানো হয়নি, আর অন্য পছন্দে প্রোগ্রাম কিছু করে না (বা অদ্‌ভুত কিছু করে)। আর সবসময়ের মতো: `scanf`-এ ভুল অক্ষর, যেমন সংখ্যার বদলে একটি **অক্ষর** লিখলে প্রোগ্রাম **পাগল হয়ে যায়**।

সতর্কতা

প্রতি সারির শেষে, অর্থাৎ x-এর for-এর পরে ও y-এর for-এর ভেতরে, **নতুন লাইনে যেতে** `printf("\n")` লাগে: না হলে সব তারকাচিহ্ন একই সারিতে **পরপর** বসে।

8.5 প্রথম ভুল: `lato` ঘোষিত নয়

প্রথম পরীক্ষায় প্রোগ্রাম **কম্পাইল হয়নি**। ক্লাসে আমি `int lato;` ঘোষণা করেছিলাম **if-এর বন্ধনীর ভেতরে**, do while-এর ঠিক আগে:



ছবি/নকশা 7: onlinegdb: `int lato;` লাইন 29-এ, `if`-এর বন্ধনীর ভেতরে। লাইন 40-এ কম্পাইলার জানায় `error: 'lato' undeclared (first use in this function)`।

একটি **বন্ধনীর ব্লকের ভেতরে** ঘোষিত ভেরিয়েবল **শুধু সেই ব্লকের ভেতরে** থাকে: ব্লক শেষ হলে ভেরিয়েবল **ধ্বংস হয়**। switch `if`-এর বাইরে, তাই সেখানে `lato` আর **নেই**, আর কম্পাইলার বলে এটি ঘোষিত নয়।

সংজ্ঞা

সংশোধন: `int lato;` `main`-এর **শুরুতে** সরাও, `int scelta;`-এর সাথে, সব ব্লকের বাইরে। এতে ভেরিয়েবল পুরো `main`-এ দেখা যায়।

8.6 প্রোগ্রাম কম্পাইল না হলে

ক্লাসে কিছু প্রোগ্রাম এই সংশোধনের পরেও কাজ করেনি। করণীয়, এই ক্রমে:

1. ভুলের **বার্তা পড়ো**: যেমন onlinegdb বলেছিল **একটি বন্ধনী নেই**।
2. **Beautify** চেপে ইন্ডেন্টেশন দেখো: একটি ক্ষেত্রে একটি `for`-এর নিচে একটি বন্ধনী **বেশি** ছিল, মুছতে হতো, আর **শেষে**, শেষ লাইনের পরে, একটি বন্ধ বন্ধনী ছিল না। ঠিক করার আগে ভালো করে পড়ো: ক্লাসে শুরুতে আমি ভুল পড়েছিলাম, একটি সঠিক বন্ধনীকে ভুল ভেবেছিলাম, আর `Ctrl+Z` দিয়ে ফেরাতে হয়েছিল। ইন্ডেন্টেশনে সঙ্গে সঙ্গে দেখা যায় কোন ব্লক বন্ধ নয়।
3. ব্লক ধরে ধরে দেখো প্রতিটি বন্ধনী **কোথায় বন্ধ হয়**: `main`-এর, `do`-র, `switch`-এর।
4. নিজের কোড লাইন ধরে ধরে সঠিক প্রোগ্রামের সাথে **মিলিয়ে দেখো** (অংশ 8.10)।

> **টীকা**
>
> **যে ভুল দেখা যায় না।** একটি ক্ষেত্রে কম্পাইলার এমন একটি লাইনে ভুল দেখাচ্ছিল যা **পুরোপুরি ঠিক** মনে হচ্ছিল, হয়তো একটি **অচেনা অক্ষরের** জন্য: ওই কোড সরালে প্রোগ্রাম চলত, ফেরালে চলত না। কোডে একটি **অদৃভুত বা অদৃশ্য** অক্ষর ঢুকলে এমন হতে পারে (যেমন অন্য জায়গা থেকে লেখা কপি করলে)। পরামর্শ: ওই অংশ **মুছে** হাতে **আবার লেখো**।

8.7 ক্ষেত্র 2: ফাঁপা বর্গ

ফাঁপা বর্গও **একটি বর্গ**: দুটি নেস্টেড for ক্ষেত্র 1 থেকে **কপি** হয়। পার্থক্য হলো কখনো **তারকাচিহ্ন** আর কখনো **ফাঁক** ছাপা হয়।

তারকাচিহ্ন ছাপা হয় শুধু **সীমানায়**, অর্থাৎ চারটি অবস্থায় (বাহু 5 হলে x ও y 0 থেকে 4):

- `x == 0`: প্রথম **কলাম**;
- `y == 0`: প্রথম **সারি**;
- `x == lato - 1`: শেষ কলাম (**বাহু বিয়োগ 1**, কারণ 5-এ নয়, 4-এ পৌঁছায়);
- `y == lato - 1`: শেষ সারি।

বাকি সব ক্ষেত্রে একটি **ফাঁক** ছাপা হয়। ফাঁক **ছাপাতেই হবে**, দেখা না গেলেও: নাহলে ডান দিকের তারকাচিহ্ন **বাঁয়ে সরে যায়**, যেমন গত পাঠের একটি অনুশীলনে হয়েছিল।

```
case 2:
    for (int y = 0; y < lato; y++)
    {
        for (int x = 0; x < lato; x++)
        {
            if ((x == 0) || (y == 0) || (x == lato - 1) || (y == lato - 1))
            {
                printf("*");
            }
            else
            {
                printf(" ");
            }
        }
        printf("\n");
    }
    nFigure++;
    break;
```

8.8 ক্ষেত্র 3: ত্রিভুজ

ত্রিভুজ আমরা পাঠ 13-এ করেছি: এটিও একটি **দ্বিগুণ নেস্টেড for**, কিন্তু x-এ যাওয়া হয় **y পর্যন্ত** (`x <= y`): প্রথম সারিতে একটি তারকাচিহ্ন, দ্বিতীয়তে দুটি, এভাবে শেষ পর্যন্ত।

```
case 3:
    for (int y = 0; y < lato; y++)
    {
        for (int x = 0; x <= y; x++)
        {
            printf("*");
        }
        printf("\n");
    }
    nFigure++;
    break;
```

8.9 default ও আকৃতির কাউন্টার

পছন্দ 1, 2, 3 বা 0 না হলে যায় `default`-এ, যা ছাপে «Scelta non valida»।

প্রোগ্রামকে আঁকা আকৃতিও **গুনতে** হবে: আরেকটি ভেরিয়েবল লাগে, `int nFigure = 0;`, যা **লুপের আগে** **শূন্য** থেকে শুরু। আকৃতির তিনটি ক্ষেত্রের প্রতিটিতে, **break-এর আগে**, 1 বাড়ে: `nFigure++;`। ব্যবহারকারী 0 বাছলে `case 0` ফলাফল ছাপে (**Le figure disegnate sono** = আঁকা আকৃতির সংখ্যা):

```
case 0:
    printf("Le figure disegnate sono %d\n", nFigure);
    break;
default:
    printf("Scelta non valida\n");
```

8.10 সম্পূর্ণ প্রোগ্রাম

এটি ক্লাসে লেখা প্রোগ্রাম, পরের বক্সের সংশোধনসহ (gcc দিয়ে পরীক্ষিত)।

```c
#include <stdio.h>

int main()
{
    int scelta;
    int lato;
    int nFigure = 0;

    do
    {
        printf("Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): ");
        scanf("%d", &scelta);

        // বাহু শুধু তিনটি আকৃতির জন্য দরকার
        if (scelta >= 1 && scelta <= 3)
        {
            do
            {
                printf("Inserisci il lato (da 1 a 9): ");
                scanf("%d", &lato);
            } while (lato < 1 || lato > 9);
        }

        switch (scelta)
        {
            case 1:
                for (int y = 0; y < lato; y++)
                {
                    for (int x = 0; x < lato; x++)
                    {
                        printf("*");
                    }
                    printf("\n");
                }
                nFigure++;
                break;
            case 2:
                for (int y = 0; y < lato; y++)
                {
                    for (int x = 0; x < lato; x++)
                    {
                        if ((x == 0) || (y == 0) || (x == lato - 1) || (y == lato - 1))
                        {
                            printf("*");
                        }
                        else
                        {
                            printf(" ");
                        }
                    }
                    printf("\n");
                }
                nFigure++;
                break;
            case 3:
                for (int y = 0; y < lato; y++)
                {
                    for (int x = 0; x <= y; x++)
                    {
                        printf("*");
                    }
                    printf("\n");
                }
                nFigure++;
                break;
            case 0:
                printf("Le figure disegnate sono %d\n", nFigure);
                break;
            default:
                printf("Scelta non valida\n");
        }

    } while (scelta != 0);

    return 0;
}
```

সতর্কতা

ক্লাসে লেখা কোডে থেকে যাওয়া দুটি ভুল (যা পরীক্ষার সময় হয়নি):

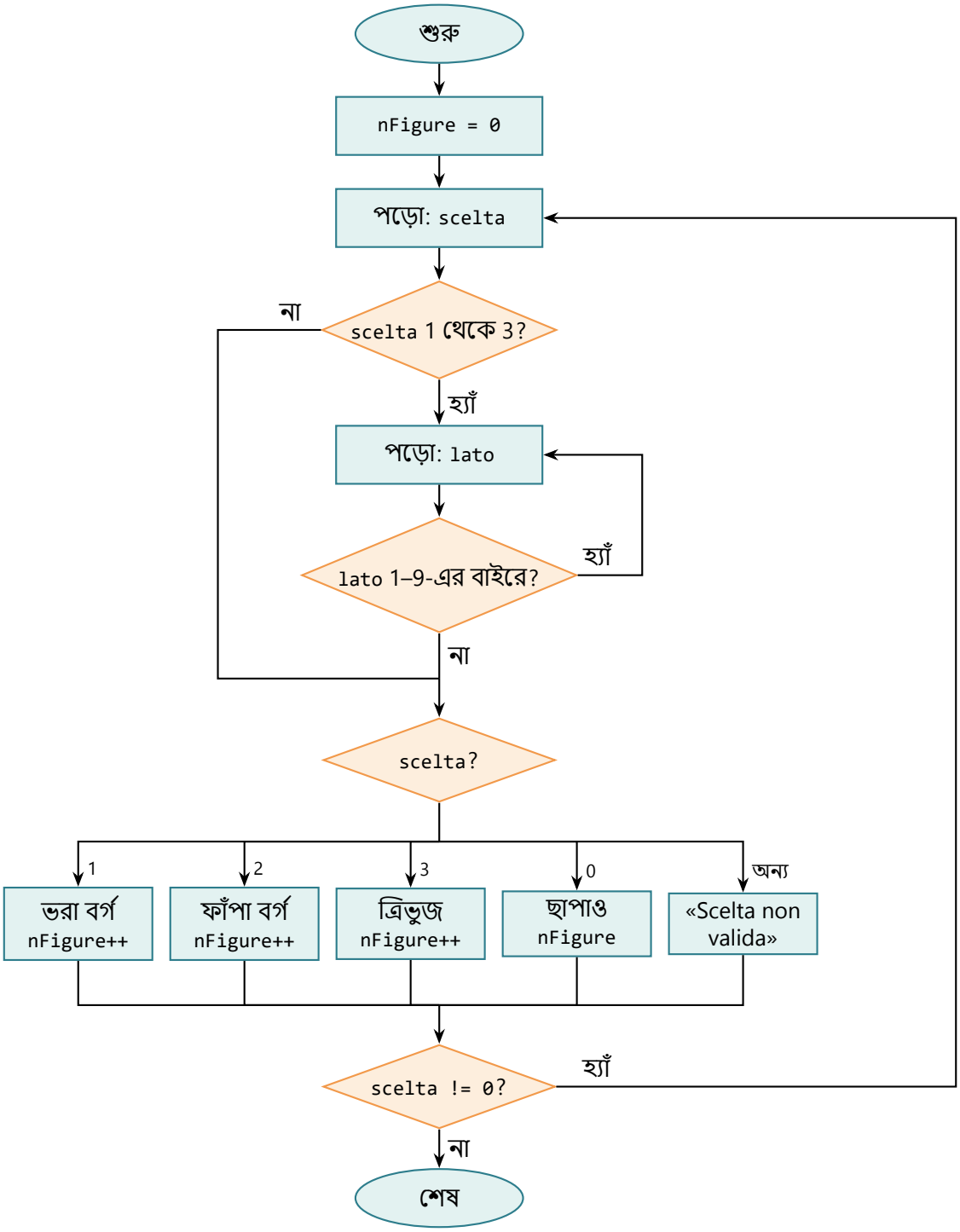
- `case 2`-এ শেষ শর্ত ছিল `(y = lato - 1)`, **একটি সমান চিহ্নে**। একটি `=` মান **নির্ধারণ করে**, **তুলনা করে না**: শর্ত সবসময় সত্য হয় (বাহু 1-এর বড় হলে) এবং `y`-ও বদলায়। বাহু 5 হলে প্রোগ্রাম ফাঁপা বর্গের বদলে তারকাচিহ্নের **দুটি ভরা সারি** ছাপে, আর কম্পাইলার **কোনো ভুল দেখায় না**। তুলনায় **দুটি সমান চিহ্ন** লাগে: `(y == lato - 1);`
- `case 2`-এ `nFigure++`-এর পরে **সেমিকোলন** ছিল না: এটি কম্পাইলার জানায়।

এছাড়া আমি `case 0` ও `default`-এর বার্তার শেষে `\n` যোগ করেছি, যাতে পরের মেনু নতুন লাইনে শুরু হয়।

8.11 একটি পরীক্ষা

```
Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): 1
Inserisci il lato (da 1 a 9): 12
Inserisci il lato (da 1 a 9): 4
****
****
****
****
Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): 2
Inserisci il lato (da 1 a 9): 5
*****
*   *
*   *
*   *
*****
Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): 3
Inserisci il lato (da 1 a 9): 4
*
**
***
****
Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): 7
Scelta non valida
Fai una scelta (1=quadrato 2=quadratoVuoto 3=triangolo 0=esci): 0
Le figure disegnate sono 3
```

8.12 ফ্লোচার্ট



ছবি/নকশা 8: আকৃতির মেনু: `switch` (রম্বস «`scelta?`») do while-এর ভেতরে, যা পছন্দ 0 থেকে আলাদা থাকা পর্যন্ত পুনরাবৃত্তি হয়।

9 এই পাঠ থেকে যা মনে রাখতে হবে

- C-তে **ইন্ডেন্টেশন** ফলাফল বদলায় না, কিন্তু কোডকে **পাঠযোগ্য** করে; **Beautify** বোতাম এটি গুছিয়ে দেয় এবং ভুল বন্ধনী খুঁজতে সাহায্য করে। আগে **গঠন** (বন্ধনী) লেখা হয়, তারপর ভরা হয়।

- **while** একটি ব্লক পুনরাবৃত্তি করে **শর্ত** **সত্য** **থাকা** **পর্যন্ত**; **for** একই কাজ করে, শুরুর মান, শর্ত ও পরিবর্তক গোল বন্ধনীতে রেখে। **কতবার** জানা থাকলে for, না জানলে while।
- **do** **while** ব্লক **অন্তত** **একবার** চালায়, শর্ত মিথ্যা হলেও; while কখনো না-ও চালাতে পারে।
- **শেষ** **সময়** মিনিট ধরে মানা হয়; বাড়ির কাজ **বাধ্যতামূলক**।
- ছকে আউটপুট আঁকা হয় **ওপরে** **বাঁ** **দিক** **থেকে**, **প্রতি** **ঘরে** **একটি** **অক্ষর**, ফাঁক **খালি**।
- **বন্ধনীর** **ব্লকের** **ভেতরে** ঘোষিত ভেরিয়েবল **শুধু** **সেখানেই** থাকে: বাইরেও দরকার হলে `main`-এর শুরুতে ঘোষণা করতে হয়।
- তুলনায় **দুটি** **সমান** **চিহ্ন** (==) লাগে; একটি সমান চিহ্ন **মান** **দেয়**, আর কম্পাইলার তা ধরে না।

বাড়ির কাজ। Classroom-এ অনুশীলনের নতুন প্যাকেট পাবে: জমা **শনিবারের** **মধ্যে**। করার আগে নিজের অনুশীলন 6 এই পাঠ্যের সম্পূর্ণ প্রোগ্রামের সাথে **মিলিয়ে** **দেখো** এবং ভুল কোথায় ছিল খুঁজে বের করো।

10 শব্দকোষ: ক্লাসে ব্যবহৃত ইতালীয় শব্দ

- **indentatura**: ইন্ডেন্টেশন
- **graffa**: কার্লি বন্ধনী
- **consegna / scadenza**: জমা / শেষ সময়
- **compito per casa**: বাড়ির কাজ
- **griglia / quadretto**: ছক / ঘর
- **quadrato pieno / vuoto**: ভরা / ফাঁপা বর্গ
- **lato**: বাহু
- **Fai una scelta**: একটি পছন্দ করো
- **esci**: বের হও
- **contatore**: কাউন্টার
- **undeclared** (ইংরেজি): ঘোষিত নয়